



Introduction to NLP

AIAA 4051 Lecture 2

Instructor: Sihong Xie

Email: sihongxie@hkust-gz.edu.cn

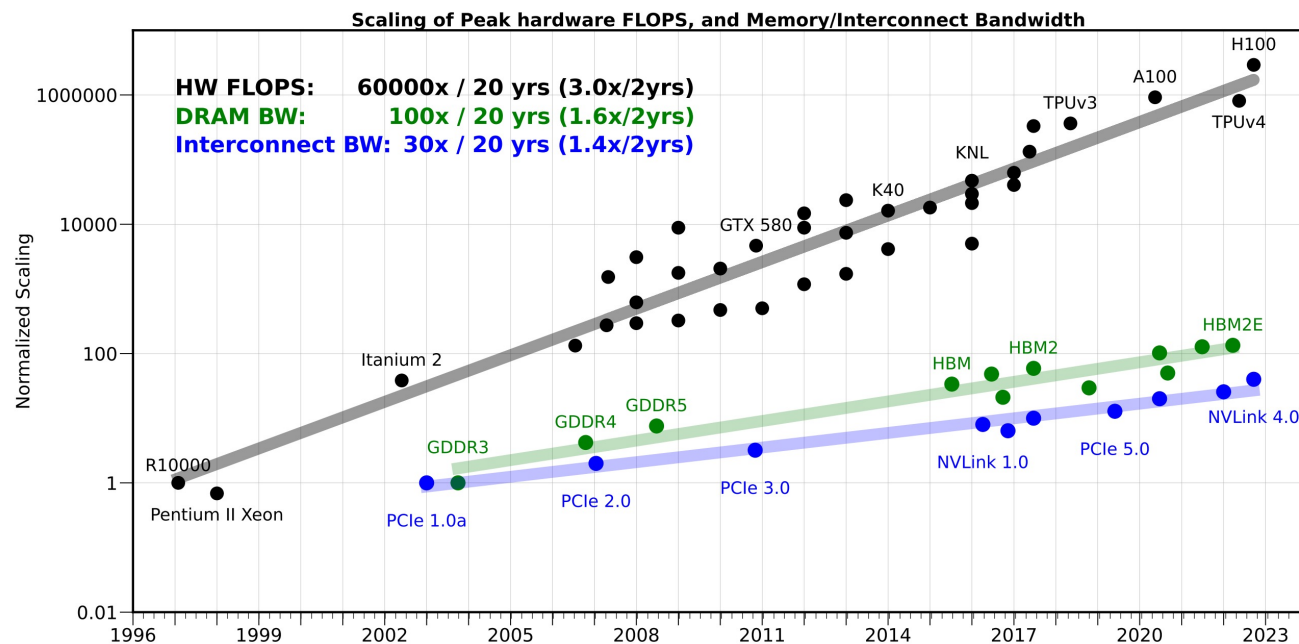
AI Thrust, HKUST(GZ)

The Efficiency Challenge

Scaling Law vs. Hardware:

As LLMs scale up, their memory and compute requirements grow faster than hardware capabilities.

- **Memory overhead:** weights for a 70B model take 140GB in FP16.
- **Memory wall:** hardware compute (FLOPS) outpaces memory bandwidth by orders of magnitude (right figure).

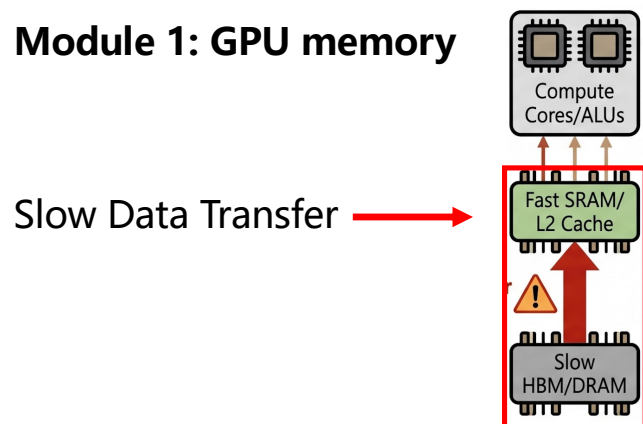


Real example: LLaMA-2 70B requires 8x A100 80GB GPUs to run in FP16 — costing ~\$20K/month on AWS. After INT4 **quantization**, it fits on a single RTX 3090 consumer GPU (~\$1,500). This is the gap compression bridges.

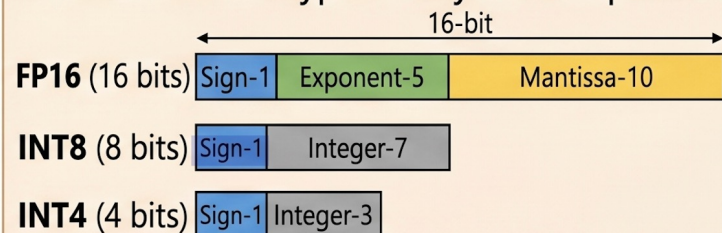
[1] Gholami, Amir, et al. "Ai and memory wall." IEEE Micro 44.3 (2024): 33-39.

Numeric formats in computers

Module 1: GPU memory



Module 2: Data Type Bit Layout Comparison



$$\text{Value} = (-1)^{\text{Sign}} \times 2^{\text{Exponent}} \times (1 + \text{Mantissa})$$

Module 3: Fixed Cache Capacity Gains (e.g., A100 40MB L2 Cache)

FP16	INT8	INT4
~20 Million Params	~40 Million Params (\$2×)	~80 Million Params (\$4×)

4x Param Capacity in On-chip Cache via INT4!

- FP16 (2 Bytes): Range [-65504, 65504]. (Example: Standard model weights)

$$N_{FP16} = \frac{40 \times 10^6 \times 8}{16} = 20 \times 10^6$$

- INT8 (1 Byte): Range [-128, 127]. (Example: Standard post-training quantization)

$$N_{INT8} = \frac{40 \times 10^6 \times 8}{8} = 40 \times 10^6$$

- INT4 (0.5 Bytes): Range [-8, 7]. (Example: Aggressive compression formats)

$$N_{INT4} = \frac{40 \times 10^6 \times 8}{4} = 80 \times 10^6$$

N: Number of parameters
C: Cache capacity in Bytes
 P_{bit} : Bit Width

$$N = \frac{C \times 8}{P_{bit}}$$

**More parameters,
but less precisions!**

Mathematical mapping

Quantization maps continuous values x to a discrete set of integers q using a Scale (S) and Zero-point (Z):

$$q = \text{clamp} \left(\text{round} \left(\frac{x}{S} + Z \right), q_{\min}, q_{\max} \right)$$

$$S = \frac{\max(\text{abs}(X))}{127} \text{ (absmax quantization), or } S = \frac{\max(X) - \min(X)}{2 * 127} \text{ (zeropoint quantization)}$$

Example (INT8 symmetric quantization):

- The range $[-\text{max}, \text{max}]$ is mapped to $[-127, 127]$.
- Quantize (FP32 to INT8):

$$q = \text{clamp} \left(\text{round} \left(\frac{2.14}{0.0275} + 0 \right), -128, 127 \right) = 78$$

- Dequantization approximates the original:

$$\hat{x} = S(q - Z) = 0.0275 \times (78 - 0) = 2.145$$

Storing less information: 8 bits rather than 32 bits,
plus quantization parameters S and Z .

Setting:

$X=2.14$ $S=0.0275$ $Z=0$

$q_{\min}=-128$ $q_{\max}=127$

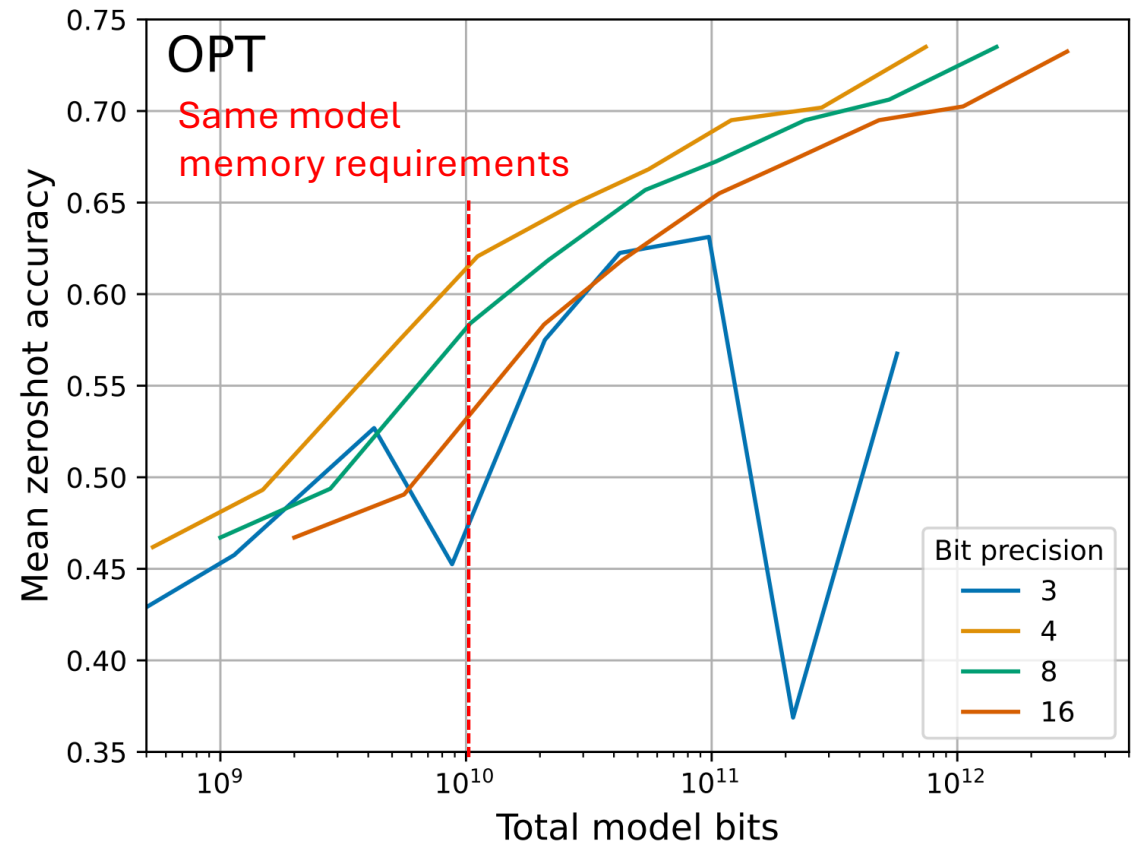


Quantization accuracy cost

The Precision Cliff

Given a fixed memory budget, how should we trade off model size versus precision?

- A larger model quantized to 4-bit yields higher accuracy than a smaller model at 8-bit or 16-bit.
- 4-bit precision strikes the optimal balance. The gain from increasing the number of parameters by 4x outweighs the quantization noises.
- Scaling breaks down at 3-bit. Quantization errors become so severe that adding more parameters can no longer recover the lost performance.



[2] Dettmers, Tim, and Luke Zettlemoyer. "The case for 4-bit precision: k-bit inference scaling laws." ICML, 2023.

| How to Quantize?

Quantization introduces errors. To minimize this accuracy drop, the industry has developed two primary pipelines.

Option 1: Quantization-Aware Training (QAT)

- What: Simulate quantization during training so the model adapts.
- Pros: Recovers accuracy perfectly, essential for 4-bit or lower.
- Cons: Expensive, requires training data and compute.

Option 2: Post-Training Quantization (PTQ)

- What: Quantize the model after it's fully trained.
- Pros: Fast, cheap, requires no training data.
- Cons: Vulnerable to the "Precision Cliff" at < 8 -bit.

[3] Liu, Zhenhua, et al. "Post-training quantization for vision transformer. NIPS 2021

[4] Nagel, Markus, et al. "Overcoming oscillations in quantization-aware training." ICML, 2022.

Quantization-Aware Training

First, we need to solve non-differentiability with the Straight-Through Estimator (STE)

Why standard training fails

Forward Pass: Simulated Quantization uses the *round()* operator to simulate low-precision formats in .

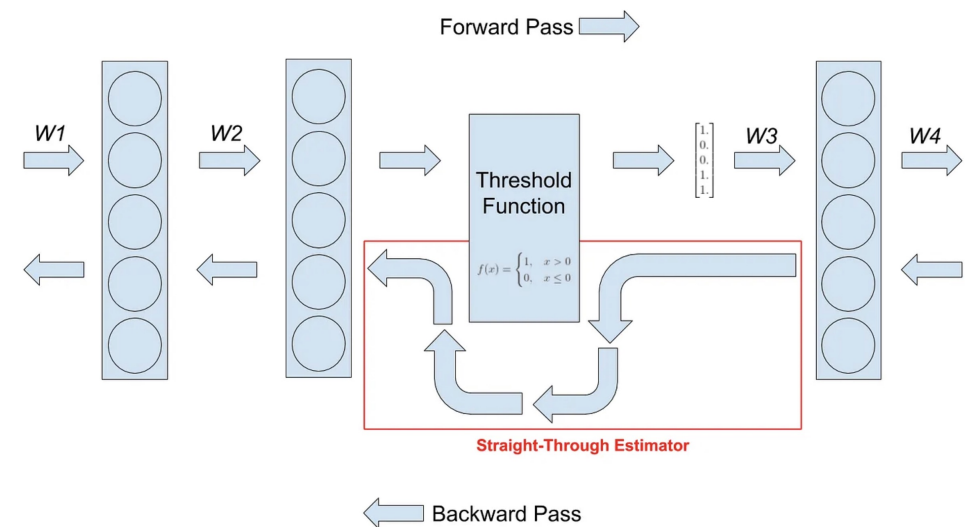
$$w_q = \text{clamp}(\text{round}(\frac{w}{S}) \times S, q_{\min}, q_{\max})$$

Try to simulate a quan/dequan process.

The Block: The derivative of *round()* is exactly **0** almost everywhere. Backpropagation fails because gradients vanish, preventing weight updates.

Straight-Through Estimator (STE)

STE acts as a “gradient bypass.” During the backward pass, it pretends the *round()* operator is an identity function (derivative = 1).



[4] Nagel, Markus, et al. "Overcoming oscillations in quantization-aware training." ICML, 2022.
Image source: <https://hassanaskary.medium.com/intuitive-explanation-of-straight-through-estimators-with-pytorch-implementation-71d99d25d9d0>

Quantization-Aware Training (QAT)

Unlike PTQ, QAT trains the model with quantized values in the forward path.

1. PTQ Initialization

Runs a small *calibration data* to compute the initial Scale s and Zero-point Z .

2. Forward Pass

Injects "*Fake Quantization*" (rounding errors) into the actual training flow.

3. Backpropagation

Uses the Straight-Through Estimator (STE) to bypass the non-differentiable round() nodes.

4. The Update Loop

Accumulates the backpropagated gradients into hidden **FP32 Shadow Weights**.

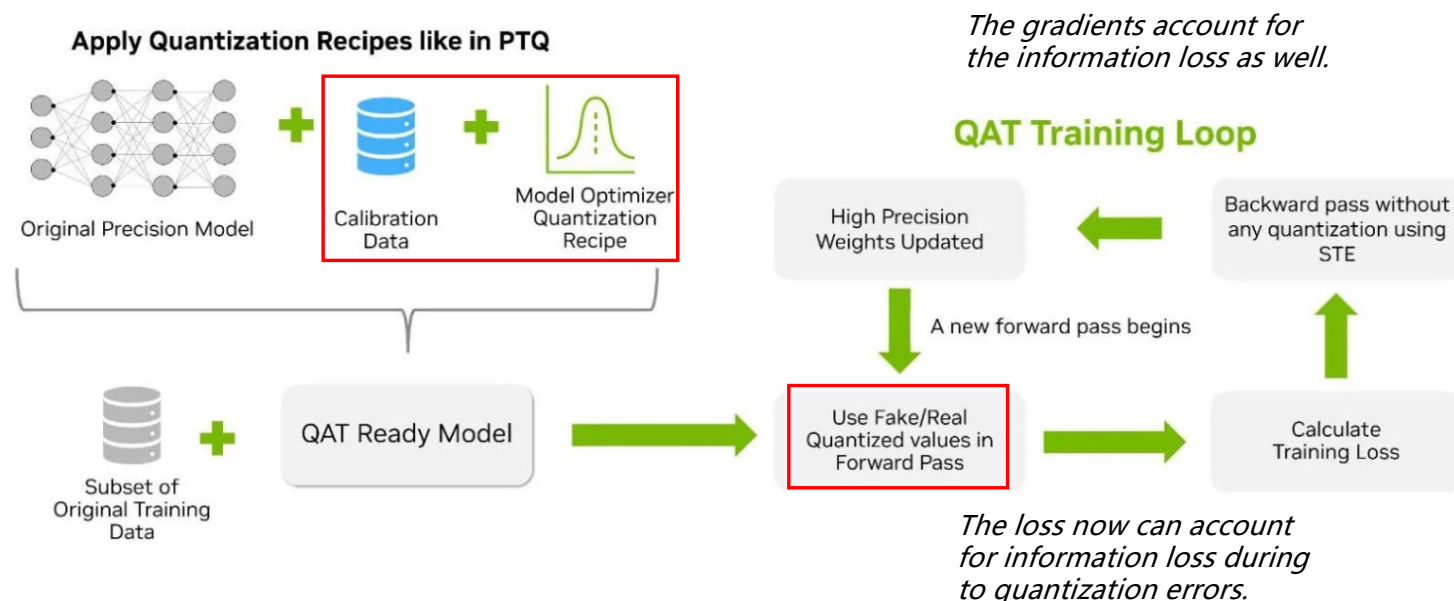


Image source: <https://developer.nvidia.cn/blog/how-quantization-aware-training-enables-low-precision-accuracy-recovery/>

Post-Training Quantization (PTQ)

Quantizing a pre-trained model without requiring any further weight updates or backpropagation.

Why 8-bit post-training quantization

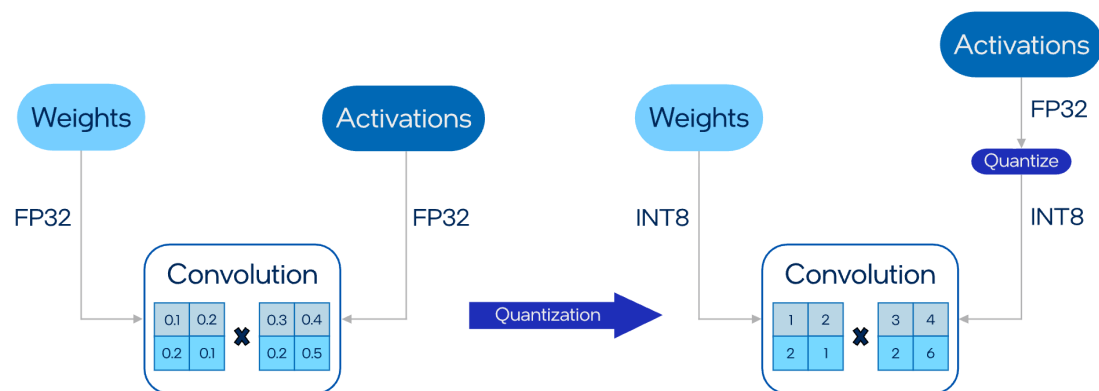
- **Memory & Speed:** Converts both weights and activations from FP32 to INT8. This shrinks the memory footprint to 1/4th of the original size and significantly accelerates matrix multiplication throughput.

Before (Left):

Standard convolution operates using high-precision.

After (Right):

1. Weights converted to INT8.
2. Activations adaptively quantized.
3. The actual convolution is executed using INT8.



[3] Liu, Zhenhua, et al. "Post-training quantization for vision transformer. NIPS 2021

Emergent outliers in LLMs

The Difficulty of LLMs

Large models (>6B) have activation channels with magnitudes 100x larger than others.

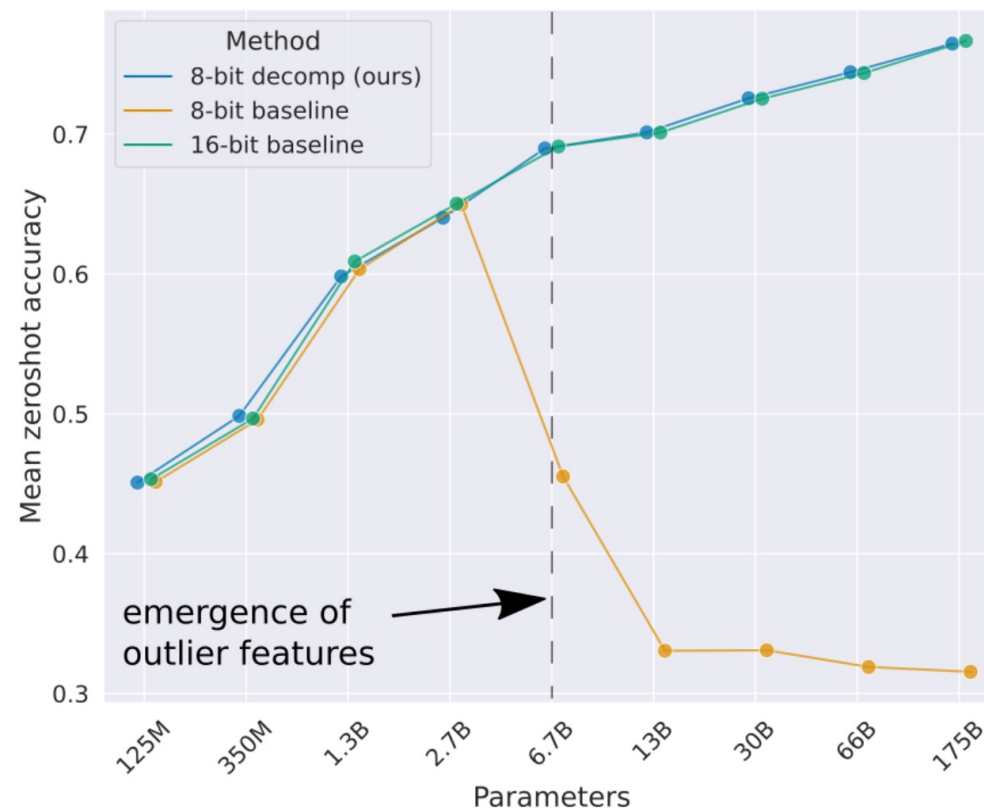
Problem: standard quantization squashes all non-outlier values to zero, destroying accuracy.

For example, in absmax quantization

$$S = \frac{\max(|X|)}{127}$$

The Squash Effect: When an outlier is 100x larger than normal values, S becomes massive. Normal values (e.g., $x=1.2$) divided by a massive S become < 0.5 , which the $\text{round}()$ function directly maps to 0.

[5] Dettmers, Tim, et al. "Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale." Advances in neural information processing systems 35 (2022): 30318-30332.

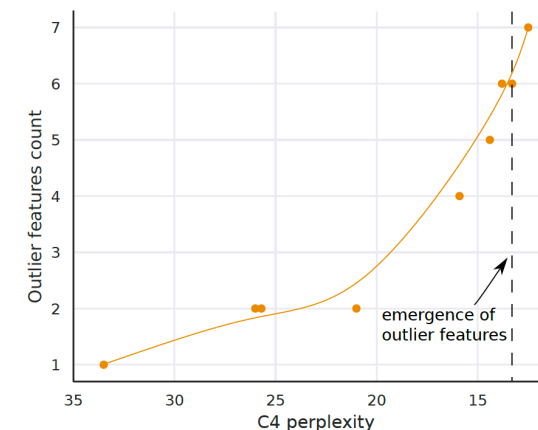
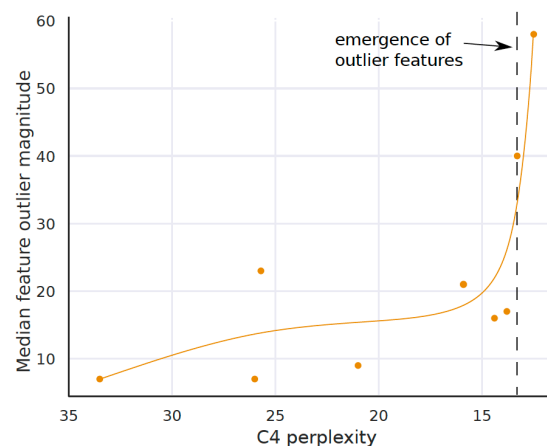
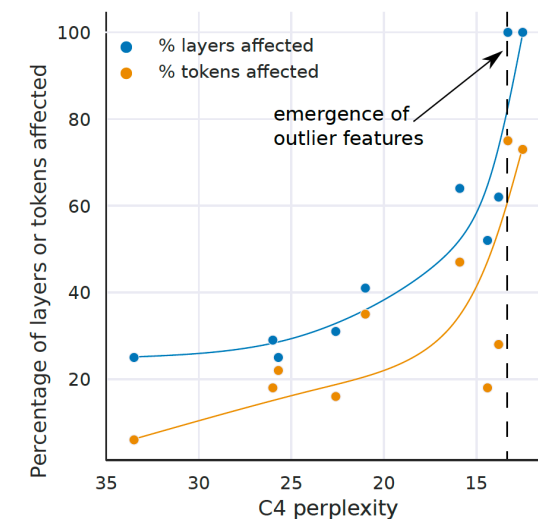
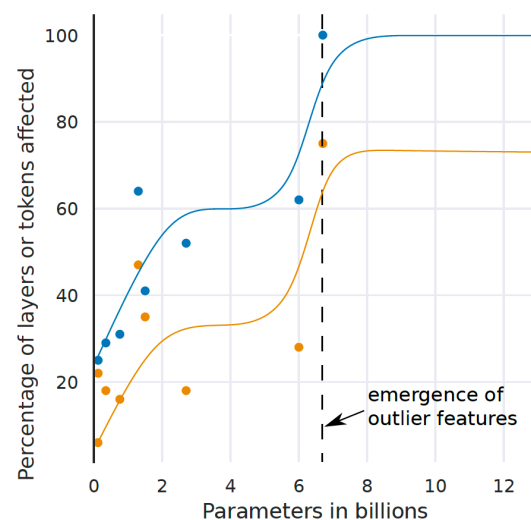


Why LLM has and needs outlier activations?

Definition (*activation outlier*): a dimension of LLM activations (hidden vectors), that has values at least 6.0, and that dimension appears in at least 25% of the layers, and at least 6% of the tokens.

Left: outlier dimensions correlates to perplexity reduction (better model fitting).

Right: outlier only appear in a few number of dimensions, leading to some rare but significant events. **Great opportunity for compression!**



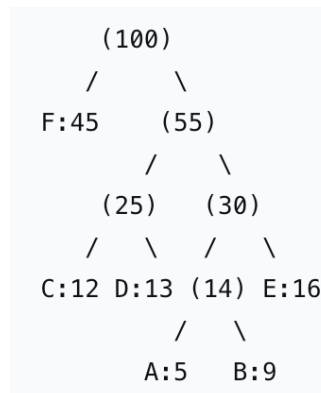
Compression efficiency

- In information theory, and coding theory in particular, the goal is to assign some discrete (quantized) codes (numbers) to symbols to reduce overall size of storing some information (e.g., piece of texts).

- Expected total length

$$L = \sum_{i=1}^k p_i \times l_i$$

- p_i : probability of seeing the i -th symbol
- l_i : code length assigned to the i -th symbol
- Want to minimize the expectation.
- Huffman tree: generates the codes.
- Average code length is close to the **information entropy** of the symbol distribution.



symbol	p	codes
a	5	1100
b	9	1101
c	12	100
d	13	101
e	16	111
f	45	0

- **Mixed precision model quantization! Rare events require longer code, while frequent events use shorter codes.**

LLM.int8 (mixed precision quantization)

Matrix multiplication can be decomposed into **independent row-column dot products**, allowing us to apply "**mixed precision**" quantization (assigning different scales for different rows of X and columns of W), instead of a single global scale (waste bits and zero outliers).

Outlier activation features

(our VIP: can be 20x larger than other dimensions)

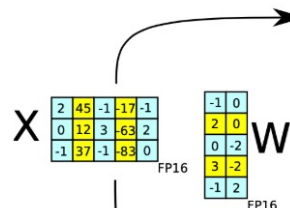
remain in FP16 but they represent only a tiny fraction of total weights.

C_x, C_w : row/column max of X/W

$X_{I8} \cdot W_{I8} = \text{Out}_{I32}$ (The Integer Core)

$(C_x \otimes C_w) / (127 \times 127)$ (The Dequantization Factor)

LLM.int8()

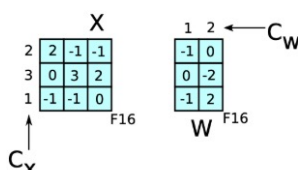


Regular values
Outliers

For frequent but small numbers

8-bit Vector-wise Quantization

(1) Find vector-wise constants: C_w & C_x



(2) Quantize

$$X_{F16} * (127/C_x) = X_{I8}$$

$$W_{F16} * (127/C_w) = W_{I8}$$

(3) Int8 Matmul

$$X_{I8} \cdot W_{I8} = \text{Out}_{I32}$$

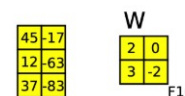
(4) Dequantize

$$\frac{\text{Out}_{I32} * (C_x \otimes C_w)}{127 * 127} = \text{Out}_{F16}$$

per-column scaling
element-wise product
per-row scaling

16-bit Decomposition

(1) Decompose outliers



(2) FP16 Matmul

$$X_{F16} \cdot W_{F16} = \text{Out}_{F16}$$

For infrequent but significant numbers (VIP!)

$\text{Out}_{I32} \times \text{Factor} = \text{Out}_{F16}$ (Precision Restoration)

[5] Dettmers, Tim, et al. "Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale." Advances in neural information processing systems 35 (2022): 30318-30332.

SmoothQuant Optimization

The Observation:

Activation (X) contains massive outliers that destroy INT8 precision.
Conversely, Weight (W) distributions are extremely flat and "easy" to quantize.

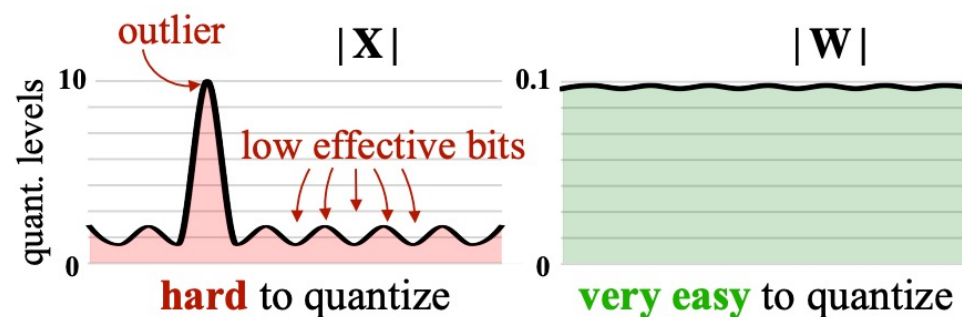
Action:

Migrate the quantization bandwidth from weights (need less bandwidth) to activations (need more bandwidth). Apply a per-channel smoothing factor s to "migrate" the quantization difficulty

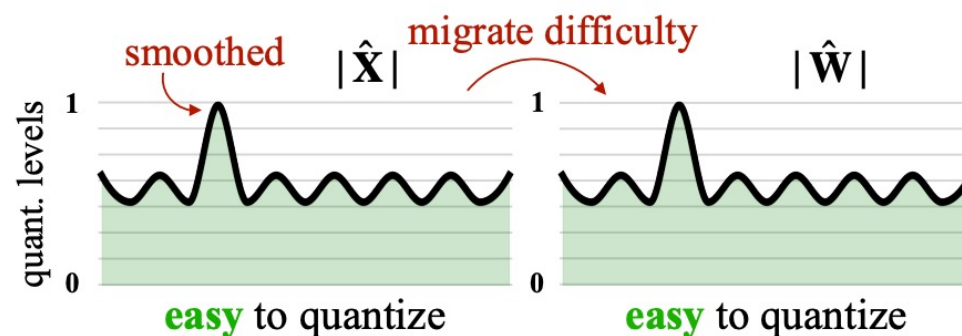
$$Y = \underbrace{(X \text{diag}(s)^{-1})}_{\text{smoother}} \cdot \underbrace{(\text{diag}(s)W)}_{\text{sharper}} = \hat{X}\hat{W}$$

The Result:

Suppress the spikes in X by pushing them into W . Since weights have "excess precision capacity," they can absorb these outliers with minimal accuracy loss.



(a) Original



(b) SmoothQuant

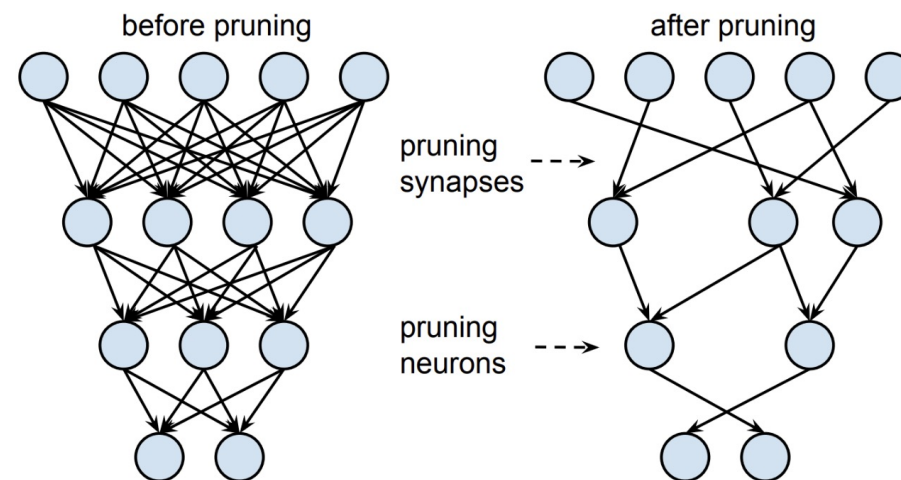
[6] Xiao, Guangxuan, et al. "Smoothquant: Accurate and efficient post-training quantization for large language models." ICLR, 2023.

Concept of Pruning

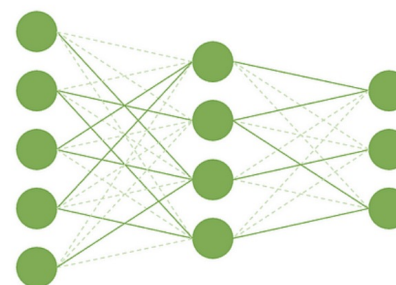
Removing Redundancy

Neural networks are often 30-50% redundant. Pruning involves identifying and zeroing out "unimportant" weights.

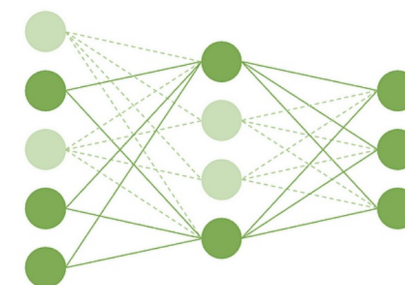
- Reduces Parameter count.
- Reduces FLOPs (if structured).
- **Unstructured:** Zeroes out individual weights based on importance regardless of their spatial location.
 - GPUs cannot efficiently skip these scattered zeros due to irregular memory access (store zeros in the sparsified vectors).
 - Reduced model size, but minimal to no wall-clock speedup on general-purpose hardware.
- **Structured:** Removes entire architectural components, such as neurons, channels, or attention heads.
 - The remaining weights form smaller, dense matrices.
 - Achieves instant, real-world speedup and significant memory bandwidth savings.



Unstructured Pruning



Structured Pruning



Activation Pruning

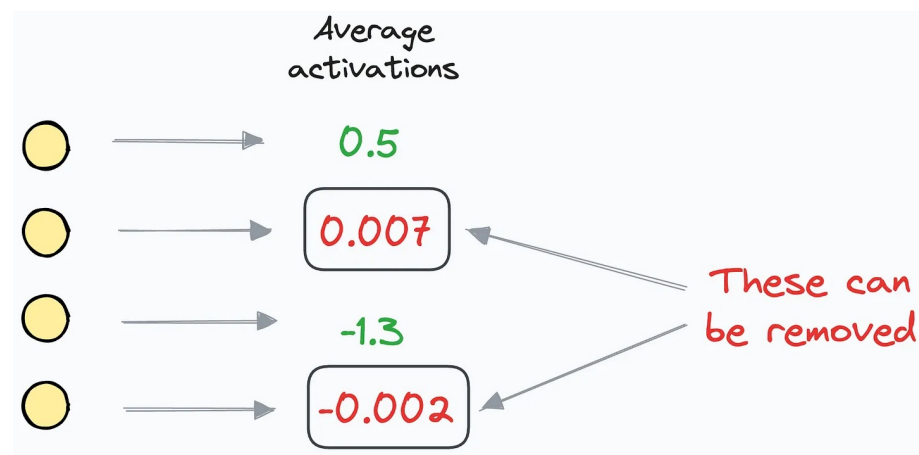
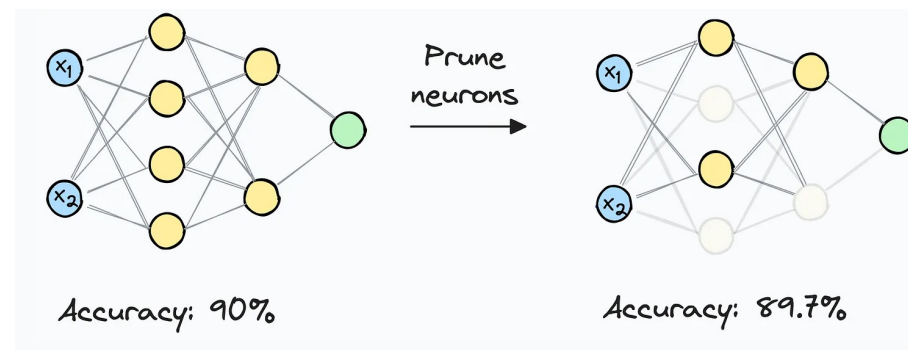
Unlike weight pruning which looks at static parameters, Activation Pruning identifies redundancy by observing the model in action.

1. We feed a representative dataset (calibration data) through the pre-trained model.
2. We monitor the output (**activations**) of every neuron across these samples. No backpropagation is required.

Green: activations with large absolute values (e.g., 0.5 and -1.3), significant features, and should be retained.

Red: activations close to 0 (e.g., 0.007 and -0.002), contributing almost nothing.

Image source: <https://blog.dailydoseofds.com/p/activation-pruning-reduce-neural>



Activation Pruning

- Example: let $w=[1,2]$, $x=[2,0.1]$, then their inner product is 2.2.
 - The first weight actually contributes more due to larger magnitude of the first activation dimension, though the first weight is less than the second one.
- Need to check both **weight** and **activation** (WANDA).
 - The significance of weight at (i,j) is determined by

$$S_{ij} = |\mathbf{W}_{ij}| \cdot \|\mathbf{X}_j\|_2$$

Statistical significance of the j -th activation dimension

- Related to the significance score used by Optimal Brain Surgeon (OBS):

$$S_{ij} = [|\mathbf{W}|^2 / \text{diag}((\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1})]_{ij}$$

$$S_{ij} \stackrel{\lambda=0}{=} \left[|\mathbf{W}|^2 / \text{diag}((\mathbf{X}^T \mathbf{X})^{-1}) \right]_{ij} \stackrel{\text{diagonal approx.}}{=} \left[|\mathbf{W}|^2 / \left(\text{diag}(\mathbf{X}^T \mathbf{X}) \right)^{-1} \right]_{ij} = (|\mathbf{W}_{ij}| \cdot \|\mathbf{X}_j\|_2)^2$$

Iterative pruning

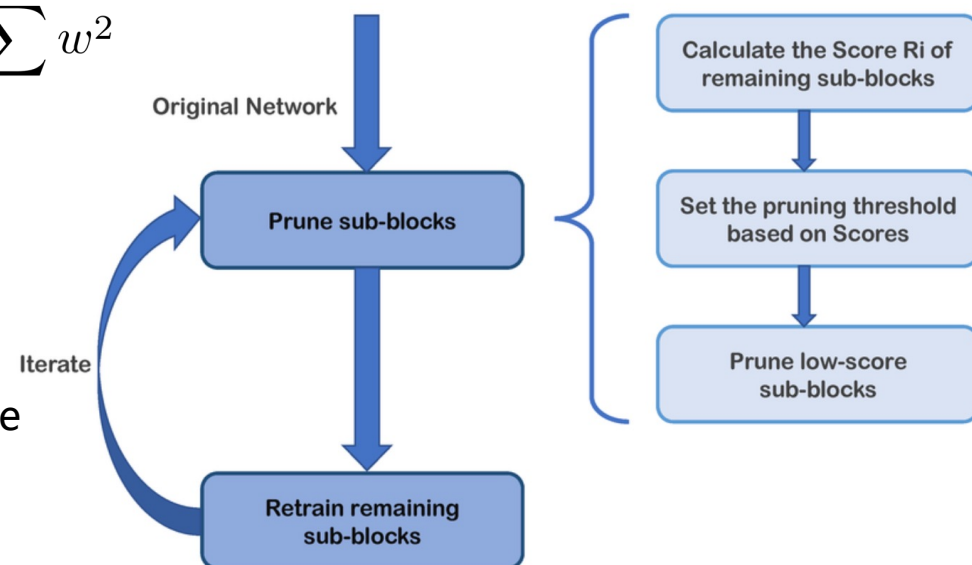
To minimize accuracy drop, pruning is performed **incrementally**. The model undergoes multiple cycles of removing redundant sub-blocks (layer by layer, channel by channel, etc.), followed by fine-tuning to recover performance.

Sub-blocks & Scores

Divide a weight matrix into non-overlapping $N \times N$ sub-blocks. Each block's importance is measured by its L2-norm magnitude:

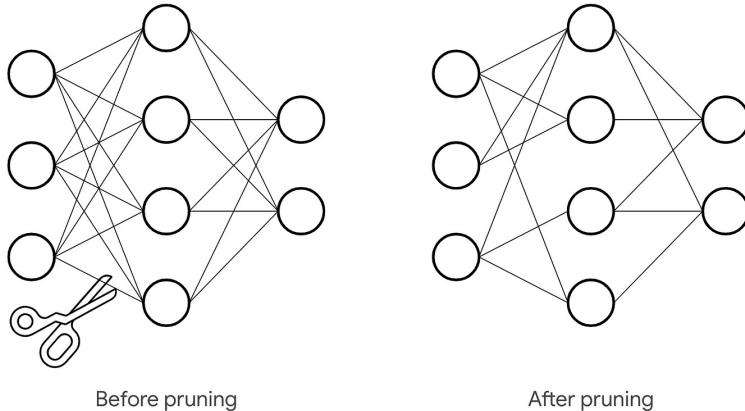
$$R_i = ||\mathbf{W}_{block_i}||_2 = \sqrt{\sum w^2}$$

1. Pruning Phase: Blocks with scores below a dynamic threshold (e.g., the weakest 10%) are masked to zero.
2. Retraining Phase: We "heal" the model by retraining the remaining dense blocks. This allows the surviving neurons to compensate for the lost information, restoring the original accuracy.
3. Iteration: This loop continues until the desired balance between model size and accuracy is reached.



Iterative pruning

Optimal Brain Surgeon: $L(\delta_p, \lambda) = \delta_p^\top \mathbf{H} \delta_p + \lambda(\mathbf{e}_p^\top \delta_p - (-w_p))$



For each layer, for each output channel,
find the optimal surgeon by solving the following:

$$w_p = \operatorname{argmin}_{w_p} \frac{(\operatorname{quant}(w_p) - w_p)^2}{[\mathbf{H}^{-1}]_{pp}}, \quad \delta_p = -\frac{w_p - \operatorname{quant}(w_p)}{[\mathbf{H}^{-1}]_{pp}} \cdot \mathbf{H}_{:,p}^{-1}$$

- Pruning one weight will impact others, so that the **δ_p vector** updates the remaining weights to minimize the impact of removing one weight.

- The big Hessian matrix per layer only needs to be calculated once during iterative pruning.

Image source: <https://matthewmcateer.me/blog/optimal-brain-damage/>

Optimal Brain Damage, NIPS 1989, Yann LeCun

Optimal Brain Surgeon and General Network Pruning. *In IEEE International Conference on Neural Networks, 1993.*

I Structured Sparsity (2:4)

The Problem: random sparsity is slow for GPUs. However, cutting entire channels (Structured) hurts accuracy too much.

The Solution: 2:4 is a "semi-structured" compromise. For every block of 4 contiguous values in a row, we force exactly 2 to be zeros.

NVIDIA designed the Sparse Tensor Core specifically to skip the zeros in this exact 2:4 pattern, yielding **2x** higher throughput while maintaining enough "granularity" to keep accuracy high.

Pros:

1. efficient memory accesses
2. a low-overhead compressed format
3. 2x math throughput increase on the NVIDIA Ampere GPU architecture

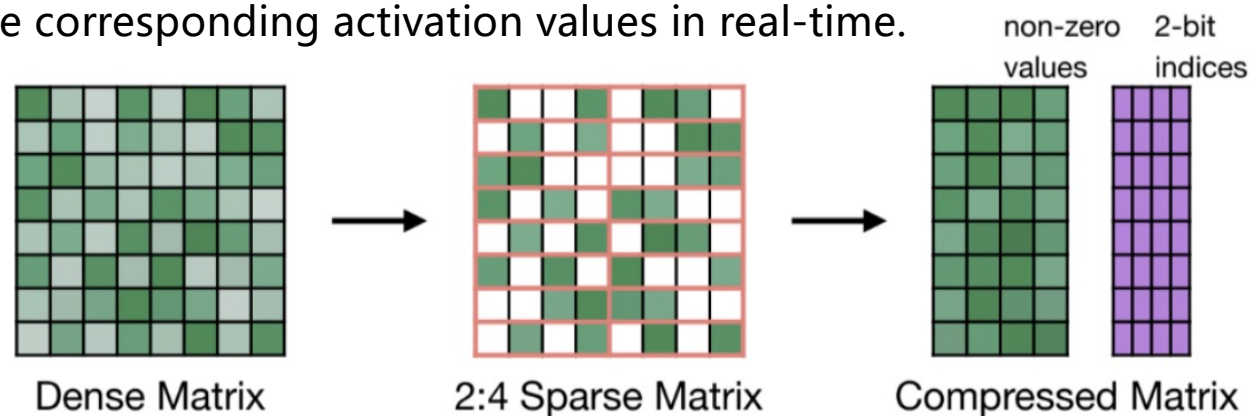
Image source: Accelerating Sparse Deep Neural Networks

Structured Sparsity (2:4)

A fine-grained 2:4 structured pruning approach to make sparsity adoption practical.

For every **block** of 4 horizontal elements, at least 2 must be zeroed out.

- **Data Matrix** ($R \times C/2$): Stores only the 2 non-zero values per block, cutting weight memory in half.
- **Indices Matrix** (Metadata): Uses 2-bit indices to store the spatial location of each non-zero element for each block.
- During inference, the Tensor Core uses the Indices to pull data from the compressed matrix and "match" it with the corresponding activation values in real-time.



Keeping highest weights ensures that the most "significant" features are preserved.

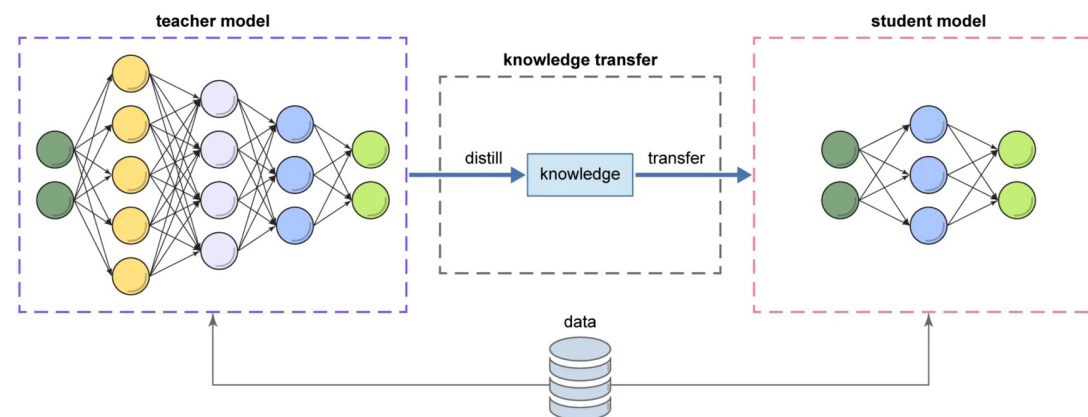
Image source: Accelerating Sparse Deep Neural Networks

Concept of Distillation

LLMs achieve SOTA performance by scaling parameters to hundreds of billions. We need the wisdom of a 70B model but the efficiency of a 7B model.

Why Distillation?

- Pruning often struggles to maintain the complex reasoning capabilities of LLMs.
- Distillation serves as a "knowledge transfer" mechanism.



Teacher-Student Framework:

- Teacher Model: A large, cumbersome, but high-performing pre-trained model.
- Student Model: A smaller, compact model that aims to mimic the Teacher's behavior.

Goal:

- Enable a small "Student" model to mimic the Teacher model's behaviors.

Image source: <https://www.britannica.com/technology/knowledge-distillation>

Concept of Distillation

Hard Targets & Soft Targets

- Hard Targets: One-hot vectors. Only tells the Student which class is correct.
- Soft Targets (Teacher's Logic): The probability distribution from the Teacher's output.
- Dark Knowledge: These tiny probabilities (e.g., 0.1% for Deer) are not random noise. They represent the Teacher's internal logic: "A horse shares structural similarities with a deer (quadruped, similar stature) but looks nothing like a car."

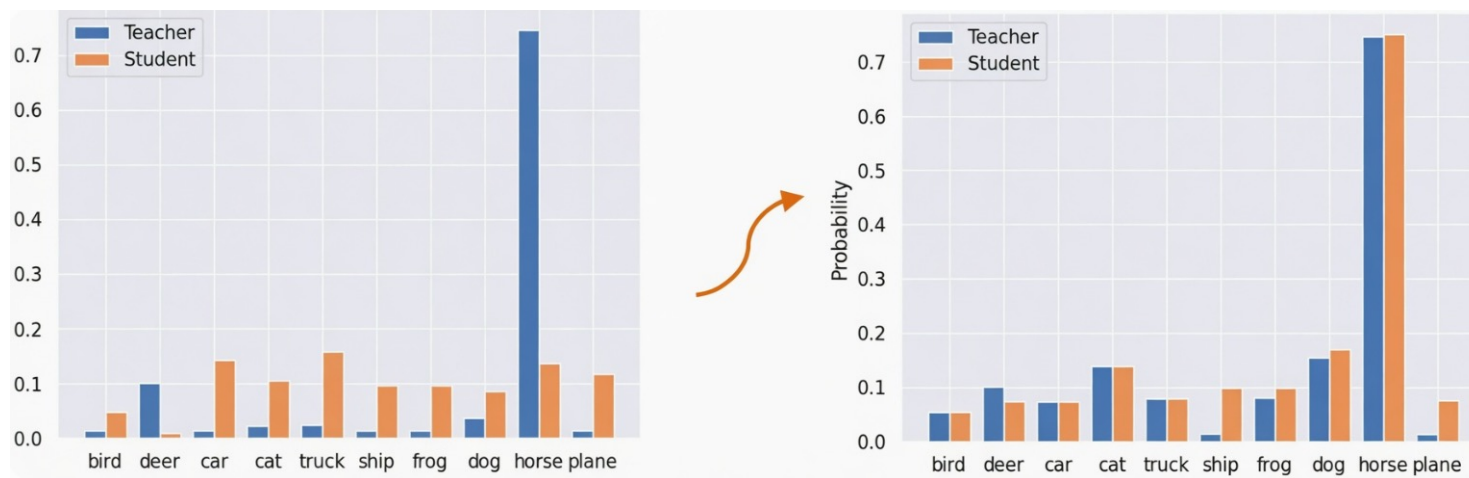


Image source:
<https://dailyinterlake.com/news/2014/mar/05/horses-and-deer-6/?=&subcategory=285%7CWine>

Distillation

The Problem:

Softmax outputs are often too “sharp” (target ≈ 1.0 , others ≈ 0.0). This extreme compression crushes the subtle differences between non-target classes to near zero.

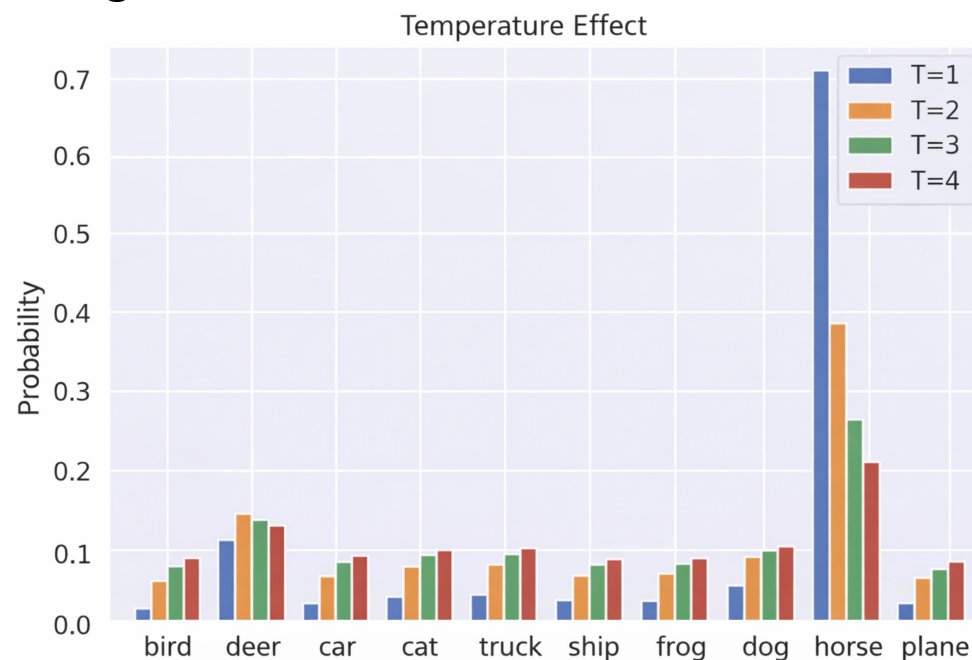
- Without Temperature:

$$q_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

- With Temperature:

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Z_i means probability of each category.

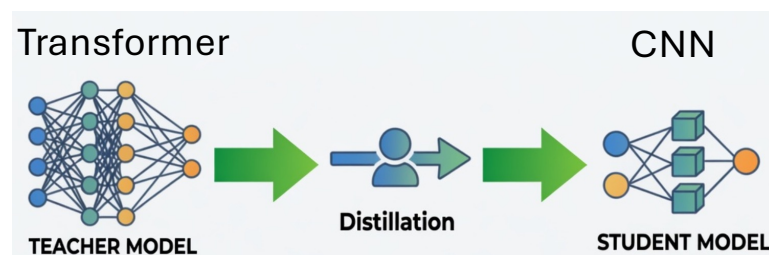


At $T=1$, the distribution has low entropy. Applying a Temperature factor ($T > 1$) scales down the logits before the exponential function, increasing distribution entropy and preventing the dominant class from saturating the gradients.

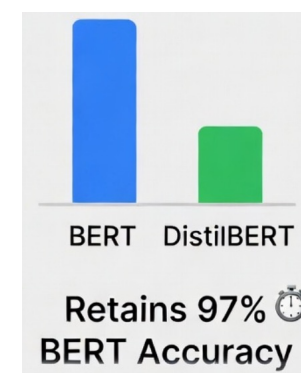
Distillation: Pros, Cons

Pros:

1. Student can be a completely different architecture^[7]



2. DistilBERT^[8]: 40% smaller, 60% faster, retains 97% of BERT accuracy.



Cons:

1. Requires training the student from scratch — expensive compared to PTQ.
2. Teacher must be available during student training — not always feasible for proprietary models.

[7] Hu, Chengming, et al. "Teacher-student architecture for knowledge distillation: A survey." arXiv preprint arXiv:2308.04268 (2023).

[8] Sanh, Victor, et al. "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter." arXiv preprint arXiv:1910.01108 (2019).